

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

Search the Help Center 

Fireblocks Help Center

Tokenization

Token contract templates

Operating the Upgradeable ERC-721F contract

Note

Tokenization is a premium feature that requires an additional purchase. If you don't have access to this feature, contact your [Customer Success Manager](#) to discuss having it enabled in your Fireblocks workspace.

Overview

The Upgradeable ERC-721F smart contract template by Fireblocks can be used to issue ERC-721 non-fungible tokens (NFTs). This token contract works seamlessly with your Fireblocks workspace and can be used to:

- Create unique NFTs like collectibles, artwork, and in-game items.
- Reflect an individual token value based on factors such as rarity, age, or attributes.
- Transfer tokens to other accounts on the Ethereum network.
- Identify who owns a token at any given point in time.
- Verify the total number of tokens created.

The contract is considered upgradeable since it is deployed as an implementation contract and a proxy contract, which points to the latest implementation. [Learn more](#).

This article describes several common operations you can use for smart contracts deployed from this template.

Deploying

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

[Learn more about smart contract roles.](#)

Minting

Before minting tokens, the Minter role must be granted to an address. A Minter role is assigned during deployment as mentioned in the previous section. However, the Default Admin role can assign additional Minter roles after the contract has been deployed.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Copy the Minter role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Minter role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

After the Minter role is granted, any address with the Minter role can mint NFTs from your deployed contract by using the mint function.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Select **Write** and use the mint function.
3. Select the source vault account.
4. Enter the address where you want to receive the NFT.
5. Enter a token ID for the NFT.
6. Enter the link to the token metadata, such as an image, in the **Uri** field. If you don't have a link, you can add it later using the tokenUriUpdate function.
7. Select **Submit**.

If the NFT was sent to a vault account in Fireblocks, it will appear in the NFT Gallery.

Burning

Before burning tokens, the Burner role must be granted to an address. Only the Default Admin role can assign the Burner role to an address.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

that should be granted the role.

5. Select **Submit**.

After the Burner role is granted, the address with the Burner role can burn tokens by using the burn function when managing the smart contract.

Updating NFT metadata

Your NFT's metadata, often a form of media, can be added when minting the token. You can also add or update the metadata later by using the tokenUriUpdate function.

Contract metadata (contractURI)

You can add and manage the metadata linked to your smart contract by using the contractUriUpdate function. For example, you may want to link your ERC-721 token to a Terms of Service, an Investment Offering, or other important documentation. We provide the Contract URI field so you can store links on the smart contract.

To store a link on the smart contract, we provide the Contract URI field where you can add those links.

First, you need to grant someone the Contract Admin role for the smart contract.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Copy the Contract Admin role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Contract Admin role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

At this point, the user granted the Contract Admin role can access the smart contract and add the link as metadata.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Select **Write** and use the contractUriUpdate function.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Token recovery

If an ERC-721F contract is used in conjunction with a Fireblocks Allowlist or Denylist, token recovery is made available. When a user holds tokens but fails the Allowlist or Denylist test, those tokens can be clawed back and returned to the smart contract.

First, you need to grant someone the Recovery role for the smart contract.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Copy the Recovery role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Recovery role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

At this point, the user granted the Recovery role can recover those tokens.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Select **Write** and use the recoverTokens function.
3. Enter the address currently holding the tokens and the amount of tokens to recover.
4. Select **Submit**.

Salvage

If a token not associated with your ERC-721F contract is accidentally transferred to it, the token will be stuck since the contract has no governance over it. You can use the Salvage role to send those external tokens back to the address that originally sent them.

First, you need to grant someone the Salvage role for the smart contract.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Copy the Salvage role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Salvage role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

3. Enter the address currently holding the tokens and the amount of tokens to send back.
 4. Select **Submit**.
-

Connect an Allowlist/Denylist

You can connect either an Allowlist or Denylist token utility contract to your ERC-721F contract. Token contracts can only be connected to a single Allowlist or Denylist. You cannot connect both to the same token contract. Learn more about our token utility contracts and how to create them.

First, you need to grant someone the Contract Admin role for the smart contract.

1. In the Fireblocks Console, go to the Tokenization page and select **More Actions (...)** > **Manage contract** for the ERC-721F contract you want the Allowlist connected to.
2. Copy the Contract Admin role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Contract Admin role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

At this point, the user granted the Contract Admin role can access the smart contract and connect the Allowlist or Denylist.

1. On the Tokenization page, copy the address to the Allowlist or Denylist.
 2. Select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
 3. Select **Write** and use the accessRegistryUpdate() function.
 4. Enter the address to the Allowlist or Denylist in the **Access registry** field.
 5. Select **Submit**.
-

Pausing/Unpausing

If needed, you can completely pause the contract from functioning. This may be useful if you believe the contract has been compromised. After resolving any security implications, you can then unpause the contract to resume functionality.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

- salvageERC20
- salvageGas
- initialize
- grantRole
- version
- supportsInterface
- contractUriUpdate

To pause the contract, you must first grant someone the Pauser role for the smart contract.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Copy the Pauser role hash from the Read functions.
3. Select **Write** and use the grantRole function.
4. Enter the hash for the Pauser role you copied in step 2, and then enter the address that should be granted the role.
5. Select **Submit**.

At this point, the user granted the Pauser can pause and unpause the contract as needed.

1. On the Tokenization page, select **More Actions (...)** > **Manage contract** corresponding to the relevant smart contract.
2. Select **Write** and use the pause() or unpause() function.
3. Select the vault account containing the address assigned the Pauser role.
4. Select **Submit**.

Audit report

- [Upgradeable ERC-721F contract audit report by OpenZeppelin](#)

Was this article helpful?

Yes

No

ON THIS PAGE

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Burning

Updating NFT metadata

Contract metadata (contractURI)

Token recovery

Salvage

Connect an Allowlist/Denylist

Pausing/Unpausing

Audit report

Related Articles



Operating the Token Sale Contract



Operating the Recurring Vesting & Payment Contract



Operating the Upgradeable ERC-20F contract



Operating the Upgradeable ERC-1155F contract



fireblocks.com

[Status](#)

[API Docs](#)

[Console](#)

info@fireblocks.com



Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)